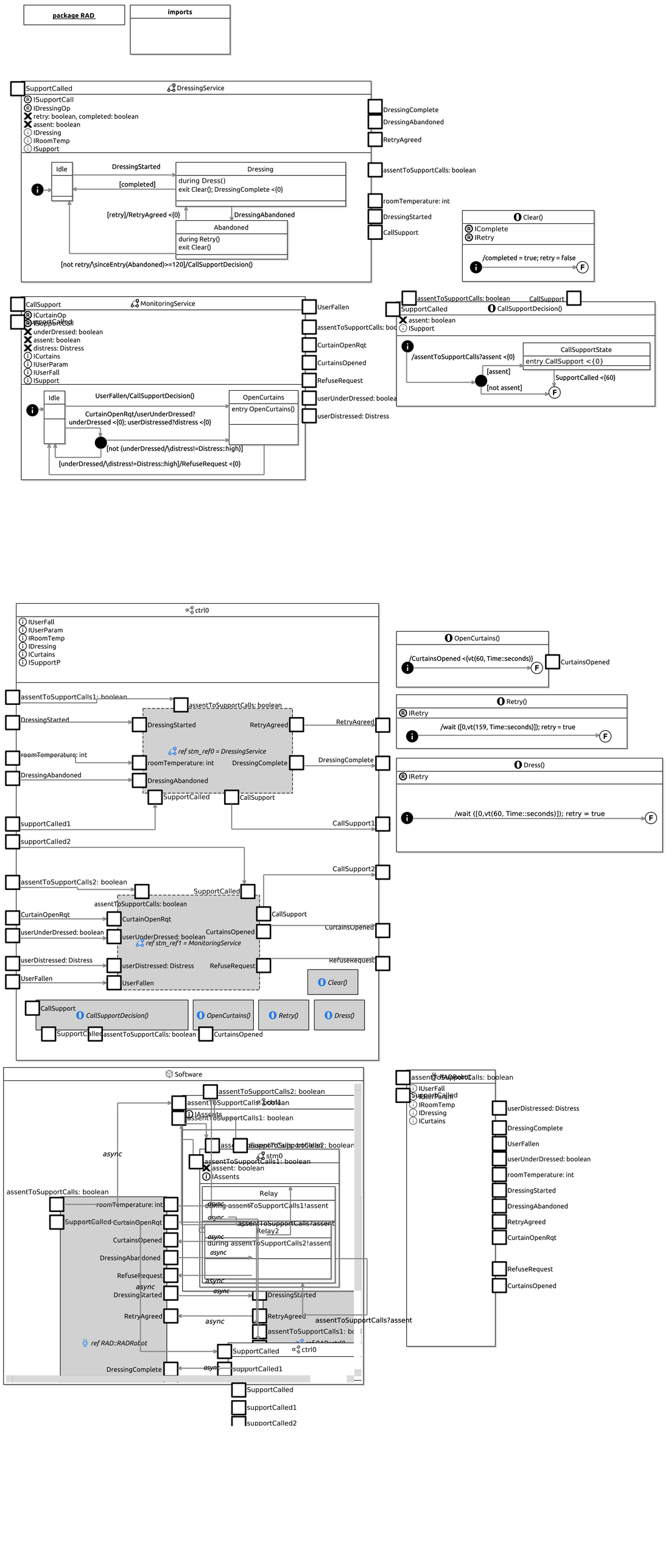
package RAD
imports
SupportCalled
DressingService
ISupportCall
IDressingOp
retry: boolean, completed: boolean
assent: boolean
IDressing
IRoomTemp
ISupport
DressingComplete
DressingAbandoned
RetryAgreed
assentToSupportCalls: boolean
roomTemperature: int
DressingStarted
CallSupport
Idle
DressingStarted
[completed]
Dressing
during Dress()
exit Clear(); DressingComplete <{0}
[retry]/RetryAgreed <{0}
DressingAbandoned
Abandoned
during Retry()
exit Clear()
[not retry/\sinceEntry(Abandoned)>=120]/CallSupportDecision()
Clear()
IComplete
IRetry
/completed = true; retry = false
CallSupport
MonitoringService
ICurtainOp
IUserFall
underDressed: boolean
assent: boolean
distress: Distress
ICurtains
IUserParam
IUserFall
ISupport
UserFallen
assentToSupportCalls: boolean
CurtainOpenRqt
CurtainsOpened
RefuseRequest
userUnderDressed: boolean
userDistressed: Distress
Idle
UserFallen/CallSupportDecision()
OpenCurtains
entry OpenCurtains()
CurtainOpenRqt/userUnderDressed?
underDressed <{0}; userDistressed?distress <{0}
[not (underDressed/\distress!=Distress::high)]
[underDressed/\distress!=Distress::high]/RefuseRequest <{0}
assentToSupportCalls: boolean
CallSupport
SupportCalled
CallSupportDecision()
assent: boolean
ISupport
/assentToSupportCalls?assent <{0}
CallSupportState
entry CallSupport <{0}
[assent]
SupportCalled <{60}
[not assent]
ctrl0
IUserFall
IUserParam
IRoomTemp
IDressing
ICurtains
ISupportP
assentToSupportCalls1: boolean
assentToSupportCalls: boolean
DressingStarted
RetryAgreed
ref stm_ref0 = DressingService
roomTemperature: int
DressingComplete
DressingAbandoned
SupportCalled
CallSupport
supportCalled1
CallSupport1
supportCalled2
CallSupport2
assentToSupportCalls2: boolean
SupportCalled
assentToSupportCalls: boolean
CurtainOpenRqt
CurtainOpenRqt
CallSupport
CurtainsOpened
CurtainsOpened
userUnderDressed: boolean
ref stm_ref1 = MonitoringService
userDistressed: Distress
RefuseRequest
RefuseRequest
UserFallen
Clear()
CallSupport
CallSupportDecision()
OpenCurtains()
Retry()
Dress()
SupportCalled
assentToSupportCalls: boolean
CurtainsOpened
OpenCurtains()
/CurtainsOpened <{vt(60, Time::seconds)}
CurtainsOpened
Retry()
IRetry
/wait ([0,vt(159, Time::seconds)]); retry = true
Dress()
IRetry
/wait ([0,vt(60, Time::seconds)]); retry = true
Software
assentToSupportCalls2: boolean
assentToSupportCalls1: boolean
IAssents
stm0
assent: boolean
IAssents
Relay
during assentToSupportCalls1!assent
assentToSupportCalls?assent
Relay2
during assentToSupportCalls2!assent
async
assentToSupportCalls: boolean
roomTemperature: int
SupportCalled
CurtainOpenRqt
CurtainsOpened
DressingAbandoned
RefuseRequest
DressingStarted
RetryAgreed
DressingComplete
DressingStarted
RetryAgreed
assentToSupportCalls?assent
assentToSupportCalls1: boolean
ref RAD::RADRobot
SupportCalled
ctrl0
supportCalled1
SupportCalled
supportCalled1
supportCalled2
IUserFall
IUserParam
IRoomTemp
IDressing
ICurtains
userDistressed: Distress
DressingComplete
UserFallen
userUnderDressed: boolean
roomTemperature: int
DressingStarted
DressingAbandoned
RetryAgreed
CurtainOpenRqt
RefuseRequest
CurtainsOpened